

# AgentGuard SDK Reference

Functions, decorators and helpers you import directly into your agent's code

---

This document describes every public function, class and decorator in the **agentguard** Python package: what it does, its exact parameters, what it returns, and a copy-pasteable example. It is written for a developer who is adding AgentGuard reliability monitoring to an existing AI agent's own code — not for the dashboard or backend internals.

Package version: **0.2.0** | Import root: **agentguard** | Requires: Python 3.10+, an AgentGuard API key for multi-user mode

# 1. Installation & Quick Start

## 1.1 Install

```
pip install agentguard
# or, from a local checkout:
pip install -e .
```

## 1.2 Get an API key

Sign up on the AgentGuard dashboard (Settings → API Keys → Create API Key), copy the raw key shown once, and store it as an environment variable. Never hard-code it in source and never expose it to frontend/browser JavaScript.

```
export AGENTGUARD_API_KEY="agp_live_..." # macOS/Linux
set AGENTGUARD_API_KEY=agp_live_...       # Windows cmd
$env:AGENTGUARD_API_KEY="agp_live_..."  # PowerShell
```

## 1.3 Minimal agent — three lines added

```
import os
from agentguard import AgentGuard

guard = AgentGuard(api_key=os.getenv("AGENTGUARD_API_KEY"), project="production")

@guard.monitor
def my_agent(task: str) -> str:
    # ... your existing agent logic, unchanged ...
    return f"handled: {task}"

my_agent("close the support ticket")
# -> a Run now appears in your AgentGuard dashboard, fully traced, evaluated,
#     risk-scored and hash-chained for audit, with zero other code changes.
```

**Note** — `AgentGuard(...)` is synchronous and does real (in-process) key resolution at construction time. Construct it once, at module scope or at the top of a synchronous `main()` — never from inside code that is already running under `asyncio.run(...)` / inside an `async def` that owns the loop. See §2.3.

## 1.4 No account yet / local & CI use

You can use AgentGuard's reliability engine (Decision Engine, evaluators, risk scoring, audit trail) with the plain, standalone decorator too — no API key, no dashboard account, single-tenant. This is what the framework has always supported and remains fully backward compatible:

```
from agentguard import monitor, Policy

@monitor(policy=Policy(max_cost=60000))
def my_agent(task: str) -> str:
    ...
```

## 2. The AgentGuard Client

```
class agentguard.AgentGuard(api_key: str | None = None, project: str = "production")
```

An authenticated handle to one workspace/project in your AgentGuard account, resolved once from an API key when it is constructed. Every run produced through this client's `.monitor` is permanently tagged with the resolved workspace and project, and is only ever visible to that account — this isolation is enforced on every dashboard/API query, not just hidden in the UI.

### 2.1 Constructor

```
AgentGuard(api_key: str | None = None, project: str = "production")
```

| Parameter            | Default                   | Description                                                                                                                                                                                                                          |
|----------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>api_key</code> | <code>None</code>         | Your AgentGuard API key ("agp_live_..."). If omitted, read from the <code>AGENTGUARD_API_KEY</code> environment variable. Raises <code>AuthenticationError</code> if neither is present, the key is unknown, or the key was revoked. |
| <code>project</code> | <code>"production"</code> | Project name within your workspace to attach runs to (e.g. "production", "staging", "development"). Created automatically on first use if it doesn't exist yet.                                                                      |

#### Raises

- `agentguard.AuthenticationError` — no key available, or the key is invalid/revoked.
- `RuntimeError` — constructed from inside an already-running event loop (see §2.3).

#### Attributes set after construction

| Name                            | What it does                                                        |
|---------------------------------|---------------------------------------------------------------------|
| <code>guard.api_key</code>      | The raw key you passed in (or read from the environment).           |
| <code>guard.user_id</code>      | The AgentGuard user account this key belongs to.                    |
| <code>guard.workspace_id</code> | The workspace this key resolved to — every run is tagged with this. |
| <code>guard.project_id</code>   | The resolved project's id (created if it didn't exist).             |

### 2.2 `guard.monitor` — attach to an agent function

```
guard.monitor(  
    func=None, *,  
    policy: Policy | None = None,  
    evaluators: list[Evaluator] | None = None,  
    llm_judge: bool | AsyncEvaluator | None = True,  
    agent_version: str | None = None,  
)
```

Identical to the standalone `agentguard.monitor` decorator (§3) — it reuses that exact same execution pipeline — except every run it produces is automatically tagged with this client's authenticated `workspace_id` / `project_id`, and the agent's function name is auto-registered as an Agent in your workspace the first time it

runs. See §3 for the full parameter reference — it is the same for both forms.

```
guard = AgentGuard(api_key=os.getenv("AGENTGUARD_API_KEY"), project="production")

@guard.monitor(policy=Policy(max_cost=60000), agent_version="v3")
async def refund_agent(order_id: str) -> str:
    ...
```

## 2.3 Async-context constraint

**Note** — `AgentGuard(...)` deliberately raises rather than silently hopping threads when called from a running event loop, because a production PostgreSQL connection pool is bound to the loop it was created on — silently sharing it across threads could corrupt it. Construct the client once, synchronously, before your async code starts:

```
# Correct
guard = AgentGuard(api_key=KEY, project="production")  # module scope, or top of main()

async def main():
    @guard.monitor
    async def agent(): ...
    await agent()

asyncio.run(main())

# Wrong – raises RuntimeError
async def main():
    guard = AgentGuard(api_key=KEY)  # <- constructed INSIDE the running loop
```

## 3. @monitor — the Core Decorator

`agentguard.monitor` (standalone) and `guard.monitor` (authenticated) wrap any agent function — sync or async — with the full reliability pipeline: run tracing, state capture, constraint/LLM evaluation, causal root-cause analysis, dynamic risk scoring, a CONTINUE / RETRY / REPLAN / HUMAN / STOP decision, per-step checkpoints, and a SHA-256 hash-chained audit trail — with zero required changes to your function's own logic.

### 3.1 Signature

```
@monitor
@monitor(
    policy: Policy | None = None,
    evaluators: list[Evaluator] | None = None,
    llm_judge: bool | AsyncEvaluator | None = True,
    agent_version: str | None = None,
)
def your_agent(...): ...           # sync or async, unchanged signature/return value
```

| Parameter                  | Default                                       | Description                                                                                                                                                                                             |
|----------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>policy</code>        | <code>Policy()</code>                         | Declarative safety boundary for this agent — max cost, forbidden actions, approval-gated actions, retry/replan bounds. See §4.                                                                          |
| <code>evaluators</code>    | <code>[ConstraintAdherenceEvaluator()]</code> | List of Evaluator instances to score each run. Pass your own to add custom, domain-specific scoring — see §8.                                                                                           |
| <code>llm_judge</code>     | <code>True</code>                             | True runs the built-in LLM-as-Judge evaluator as a background task after the run (never on the critical path). False disables it. Pass a custom AsyncEvaluator instance to use your own judge/provider. |
| <code>agent_version</code> | <code>None</code>                             | Optional label (e.g. "v2", "prompt-B") stamped on every run — used by Regression Comparison / A-B analysis on the dashboard. Purely informational.                                                      |

### 3.2 What it guarantees

- **Never changes your return value or swallows your exceptions** — every exception your function raises still propagates to the caller after being recorded, except the control-flow signals below, which the decorator interprets itself.
- **Sync and async both supported.** A sync function is internally run via `asyncio.run(...)` — so it cannot itself already be inside a running loop, and cannot call the async-only primitives (`perform_action/request_approval`). Define your agent as `async def` if you need those.
- **Backward compatible** — a plain `@monitor` with no arguments behaves exactly like the framework's original release: nothing here is opt-out, everything extra is opt-in from your agent's own code.

### 3.3 The execution loop your function can drive

From inside a monitored function, you can raise these to control what AgentGuard does next (all defined in `agentguard.errors` — full reference in §6):

| Name                                                                   | What it does                                                                                                                                                            |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>raise TransientError(msg)</code>                                 | Bounded RETRY — the SAME call is retried, up to <code>Policy.retry_limit</code> times.                                                                                  |
| <code>raise ReplanRequested(reason)</code>                             | Bounded REPLAN — your function is called again with a fresh attempt budget; <code>agentguard.get_replan_context()</code> returns your reason/context on that next call. |
| <code>await<br/>agentguard.perform_action(name,<br/>**evidence)</code> | Synchronous policy guardrail + optional live human-approval gate for one named action — see §5.                                                                         |

### 3.4 Full example

```

from agentguard import monitor, Policy
from agentguard.errors import TransientError, ReplanRequested

@monitor(policy=Policy(max_cost=60000, retry_limit=2, max_replans=1))
async def research_agent(query: str) -> str:
    try:
        result = call_search_api(query)
    except TimeoutError as exc:
        raise TransientError(f"search API timed out: {exc}")

    if not result:
        raise ReplanRequested("search returned nothing, trying a narrower query")

    return summarize(result)

```

## 4. Policy — Declarative Safety Boundaries

`class agentguard.Policy` (a Pydantic model) — the safety contract you attach to a monitored agent via `@monitor(policy=...)`. Every field has a safe default, so `Policy()` alone is valid.

| Parameter                         | Default               | Description                                                                                                                                                                                                       |
|-----------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>max_cost</code>             | <code>None</code>     | A numeric budget ceiling. The default evaluator ( <code>ConstraintAdherenceEvaluator</code> ) checks this against your agent's reported state (see <code>AgentState</code> 's <code>max_budget</code> field, §5). |
| <code>require_approval</code>     | <code>[]</code>       | List of action names that must be approved by a human before proceeding — see <code>agentguard.perform_action</code> (§5).                                                                                        |
| <code>forbidden_actions</code>    | <code>[]</code>       | List of action names that immediately raise <code>ForbiddenActionError</code> the instant your agent attempts them — no human round-trip, purely local/synchronous.                                               |
| <code>default_on_uncertain</code> | <code>"stop"</code>   | <code>"continue"   "stop"   "human"</code> — fallback behavior when an evaluator's own confidence is too low to trust its verdict.                                                                                |
| <code>retry_limit</code>          | <code>2</code>        | Max <code>RETRY</code> transitions (from <code>TransientError</code> ) before falling through to <code>on_retry_exhausted</code> .                                                                                |
| <code>on_retry_exhausted</code>   | <code>"replan"</code> | <code>"replan"   "stop"</code> — what happens once <code>retry_limit</code> is hit.                                                                                                                               |
| <code>max_replans</code>          | <code>1</code>        | Max <code>REPLAN</code> transitions (from <code>ReplanRequested</code> , or from exhausted retries if <code>on_retry_exhausted="replan"</code> ) before the run is <code>STOPped</code> .                         |
| <code>human_timeout_s</code>      | <code>120.0</code>    | Seconds to wait for a human approval decision before treating it as a denial. The agent never hangs forever.                                                                                                      |
| <code>on_tool_exhausted</code>    | <code>"replan"</code> | <code>"replan"   "human"</code> — what <code>agentguard.call_tool()</code> does when a tool and its fallback both fail (§7).                                                                                      |
| <code>version</code>              | <code>1</code>        | Increment by hand whenever you change what a named policy means. Every audit event for a run permanently records the policy version active when that run started.                                                 |

### Example

```
from agentguard import Policy

policy = Policy(
    max_cost=60000,
    forbidden_actions=["drop_database", "wire_transfer_over_10k"],
    require_approval=["refund", "send_customer_email"],
    default_on_uncertain="human",
    retry_limit=3,
    max_replans=2,
    human_timeout_s=90,
)
```

## 5. Reporting State & Guarded Actions

These functions operate on an ambient "current run" context — call them from anywhere inside a function wrapped by `@monitor` / `@guard.monitor`, with no object to thread through your own code. Calling any of them outside a monitored run raises `RuntimeError`.

### 5.1 `get_state()`

```
agentguard.get_state() -> AgentState
```

Returns the mutable state object for the run currently executing. Read any field you or AgentGuard has set (e.g. after a `Policy(max_cost=...)`, `state.max_budget` is pre-seeded for you).

### 5.2 `update_state(**kwargs)`

```
agentguard.update_state(**kwargs) -> None
```

Merges the given keyword arguments into the run's state, AND records a new, timestamped state snapshot (S1, S2, S3, ...) that the causal Root-Cause Engine diffs if the run later fails a constraint — this is how AgentGuard tells you exactly which step introduced a problem.

```
from agentguard import update_state

@monitor(policy=Policy(max_cost=60000))
def budget_agent(task):
    update_state(max_budget=60000, spent_so_far=0)
    for step in plan(task):
        cost = execute(step)
        update_state(spent_so_far=get_state().spent_so_far + cost)
    return "done"
```

### 5.3 `reset_state(**kwargs)`

```
agentguard.reset_state(**kwargs) -> None
```

Replaces the ENTIRE state with a fresh one (only the given kwargs survive) — models a real failure mode where an agent's working memory is silently truncated or clobbered between steps (e.g. a context-window trim). Also records a new snapshot.

### 5.4 `record_tokens(...)`

```
agentguard.record_tokens(
    *, model_name: str | None = None,
    input_tokens: int | None = None,
    output_tokens: int | None = None,
    cost_usd: float | None = None,
) -> None
```

Reports REAL LLM token usage/cost for the Tokens & Cost dashboard page. Call this only with numbers you actually have from your LLM provider's response — there is no built-in estimate. A run that never calls this simply shows "N/A" on the dashboard, never a fabricated number. Multiple calls in the same run accumulate (input/output tokens and cost sum; `model_name` is overwritten by the latest call).

```
response = openai_client.chat.completions.create(...)
agentguard.record_tokens(
    model_name="gpt-4o",
    input_tokens=response.usage.prompt_tokens,
    output_tokens=response.usage.completion_tokens,
    cost_usd=estimate_cost(response.usage),
)
```

## 5.5 `perform_action(action, **evidence)`

```
agentguard.perform_action(action: str, **evidence) -> PolicyFinding | None
```

Declares that your agent is about to take a named action, in two steps: (1) an immediate, local, synchronous check against `Policy.forbidden_actions` — raises `ForbiddenActionError` right away if listed, with no I/O; (2) if the action is listed in `Policy.require_approval`, *awaits* a live human decision (see 5.6) before returning. Must be awaited — only usable from an `async def` agent.

```
from agentguard import perform_action
from agentguard.errors import ForbiddenActionError, HumanRejected

@monitor(policy=Policy(
    forbidden_actions=["drop_database"],
    require_approval=["refund"],
))
async def support_agent(ticket):
    if ticket.wants_refund:
        try:
            await perform_action("refund", amount=ticket.amount, customer=ticket.customer_id)
        except HumanRejected:
            return "refund denied by reviewer"
        issue_refund(ticket)
    return "handled"
```

## 5.6 `request_approval(...)` — low-level primitive

```
agentguard.request_approval(
    action: str, *, finding: PolicyFinding | None = None,
    reason: str = "", evidence: dict | None = None,
) -> HumanDecision
```

What `perform_action` calls internally to publish a pending approval request to the dashboard/WebSocket channel and await a human's response, subject to `Policy.human_timeout_s`. Raises `HumanRejected` on rejection or timeout (timeout always denies — the agent never hangs indefinitely), raises `HumanReplanRequested` if the reviewer asks for a re-plan instead, or returns the resolved `HumanDecision` on approval. Most agent code should call `perform_action` instead — use this directly only if you need a human gate with no policy-forbidden check attached to it.

## 5.7 get\_replan\_context()

```
agentguard.get_replan_context() -> dict
```

On the invocation of your agent function that follows a REPLAN transition, returns the reason/context that triggered it (empty dict on a run's first attempt) — read this at the top of your function to change your plan instead of blindly repeating it.

```
@monitor(policy=Policy(max_replans=2))
async def planner_agent(task):
    ctx = agentguard.get_replan_context()
    if ctx:
        task = f"{task} (avoid: {ctx.get('reason')})"
    ...
```

## 6. Exceptions Your Agent Code Raises

Import from `agentguard.errors`. These are how your agent communicates a non-success outcome to the Decision Engine — raise them from inside a monitored function instead of returning a special value or swallowing the failure yourself.

| Name                                                  | What it does                                                                                                                                                                                   |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TransientError(message, **evidence)</code>      | "Retry me." Caught by @monitor's RETRY path, bounded by <code>Policy.retry_limit</code> . Extra kwargs are stored as structured evidence on the audit trail.                                   |
| <code>ReplanRequested(reason, *, context=None)</code> | "My plan is no longer valid." Caught by @monitor's REPLAN path, bounded by <code>Policy.max_replans</code> . context is handed back via <code>get_replan_context()</code> on the next attempt. |
| <code>ForbiddenActionError(action)</code>             | Raised automatically by <code>perform_action()</code> for an action listed in <code>Policy.forbidden_actions</code> — you normally don't raise this yourself, but you may catch it.            |
| <code>HumanRejected(action, reason)</code>            | Raised by <code>perform_action()/request_approval()</code> when a human denies the action (or it times out). Catch this to handle a denied action gracefully instead of letting the run STOP.  |
| <code>HumanReplanRequested(action, reason)</code>     | A subclass of <code>ReplanRequested</code> raised when a human reviewer asks for a re-plan instead of approving/denying — handled by the same REPLAN path.                                     |

## 7. Tool Calls & Reliability

Optional, advanced: wrap your agent's tool invocations through AgentGuard's tool registry to get per-tool reliability tracking (success rate, latency, timeout rate) and automatic fallback to an explicitly registered alternative tool when the primary tool is failing.

### 7.1 Register your tools once at startup

```
from agentguard.tools.registry import get_registry

get_registry().register_tool(name: str, fn: Callable) -> None
get_registry().register_alternative(
    primary: str, fallback: str, *, reliability_threshold: float = 0.8,
) -> ToolAlternative
```

```
from agentguard.tools.registry import get_registry

registry = get_registry()
registry.register_tool("search_primary", call_primary_search_api)
registry.register_tool("search_backup", call_backup_search_api)
registry.register_alternative("search_primary", "search_backup", reliability_threshold=0.8)
```

**Note** — AgentGuard never substitutes a tool that wasn't explicitly registered as an alternative — there is no automatic/implicit fallback to an arbitrary tool.

### 7.2 Call a registered tool from your agent

```
from agentguard.context import call_tool

await call_tool(
    tool_name: str, *args,
    primary_attempts: int = 1,
    timeout_s: float | None = None,
    **kwargs,
) -> Any
```

Invokes the named tool, recording latency/outcome/duplicate-call detection for every attempt. If the primary tool fails `primary_attempts` times, AgentGuard checks the tool's historical reliability profile; if a fallback is registered and the primary is unreliable or has insufficient history, it tries the fallback once. If both fail (or no fallback exists), it raises `ReplanRequested` — or escalates to a human approval, per `Policy.on_tool_exhausted` — so your existing REPLAN/HUMAN handling in the monitored function takes over; it never invents a new control-flow path.

```
from agentguard.context import call_tool

@monitor(policy=Policy(on_tool_exhausted="replan", max_replans=1))
async def search_agent(query):
    results = await call_tool("search_primary", query, primary_attempts=2, timeout_s=5.0)
    return summarize(results)
```

## 8. Custom Evaluators

Every run is scored by a list of Evaluator instances (default: just the built-in ConstraintAdherenceEvaluator). Add your own domain-specific scoring by subclassing and passing it in via `@monitor(evaluators=[...])`.

```
from agentguard.evaluators.base import Evaluator
from agentguard import EvalResult, Run

class MyEvaluator(Evaluator):
    name = "my_custom_check"

    def evaluate(self, run: Run) -> EvalResult:
        # inspect run.initial_state / run.final_state / run.policy / run.actions
        passed = ...
        return EvalResult(
            evaluator=self.name, passed=passed, score=1.0 if passed else 0.0,
            label="pass" if passed else "fail", confidence=0.9, reason="...",
        )
```

```
from agentguard import monitor, Policy
from agentguard.evaluators.rule_based import ConstraintAdherenceEvaluator

@monitor(policy=Policy(max_cost=60000), evaluators=[
    ConstraintAdherenceEvaluator(), # keep the built-in check too
    MyEvaluator(),
])
def my_agent(task): ...
```

This runs synchronously as part of the monitored call. For a slower, asynchronous check (e.g. an LLM-as-judge call to a model), pass your own AsyncEvaluator via the decorator's `llm_judge=` parameter instead — it runs as a background task and never blocks your agent's return value; if it later flags the run unsafe, AgentGuard records a fresh STOP decision after the fact rather than mutating the original one.

## 9. Recovery & Rollback (Advanced)

Rollback and replay are dashboard/operator-invoked operations, not something your agent's own code calls during a normal run. If you are building an automated recovery workflow around AgentGuard (rather than using the dashboard's Rollback button), the primitive your own script uses to start a fresh, recovered run is:

```
from agentguard.recovery.seed import seed_recovery_state

seed_recovery_state(
    state: dict, *, parent_run_id: str | None = None, checkpoint_id: str | None = None,
) -> None
```

```
result = await agentguard.recovery.rollback(repository, run_id, "S2")
agentguard.recovery.seed_recovery_state(
    result.restored_state, parent_run_id=run_id, checkpoint_id=result.checkpoint.id,
)
recovered = await safe_agent("retry the task") # a normal @monitor call, seeded from S2
```

**Note** — This is a one-shot signal consumed by the very next `@monitor` call on the current context — it never leaks into an unrelated later run, and a normal call made without seeding is completely unaffected.

## 10. Quick Reference — Everything At a Glance

### Import from agentguard (top-level, everyday use)

| Name                                                        | What it does                                                             |
|-------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>AgentGuard(api_key=None, project="production")</code> | Authenticated client — construct once, use <code>.monitor</code> .       |
| <code>AuthenticationError</code>                            | Raised by <code>AgentGuard()</code> for a missing/invalid/revoked key.   |
| <code>monitor</code>                                        | The core decorator — standalone, single-tenant form.                     |
| <code>Policy(...)</code>                                    | Declarative safety boundary passed to <code>monitor(policy=...)</code> . |
| <code>get_state()</code>                                    | Read the current run's mutable state.                                    |
| <code>update_state(**kwargs)</code>                         | Merge into state + record a new diffable snapshot.                       |
| <code>reset_state(**kwargs)</code>                          | Replace the entire state + record a new snapshot.                        |
| <code>record_tokens(...)</code>                             | Report real LLM token/cost usage for this run.                           |
| <code>perform_action(action, **evidence)</code>             | Guardrail check + optional human-approval gate.                          |
| <code>request_approval(...)</code>                          | Low-level human-approval primitive.                                      |
| <code>get_replan_context()</code>                           | What the last REPLAN handed back to this attempt.                        |
| <code>wait_for_background_tasks()</code>                    | Test/CLI helper: await pending background LLM-judge tasks.               |
| <code>configure(repository)</code>                          | Advanced/test-only: point the SDK at a specific storage backend.         |

### Import from agentguard.errors

| Name                                               | What it does                                                    |
|----------------------------------------------------|-----------------------------------------------------------------|
| <code>TransientError(message, **evidence)</code>   | Raise for a bounded RETRY.                                      |
| <code>ReplanRequested(reason, context=None)</code> | Raise for a bounded REPLAN.                                     |
| <code>ForbiddenActionError(action)</code>          | Raised internally by <code>perform_action()</code> ; catchable. |
| <code>HumanRejected(action, reason)</code>         | Raised when a human denies / a request times out.               |
| <code>HumanReplanRequested(action, reason)</code>  | Raised when a human asks for a re-plan.                         |

### Other advanced imports

| Name                                                           | What it does                                       |
|----------------------------------------------------------------|----------------------------------------------------|
| <code>agentguard.context.call_tool(...)</code>                 | Reliability-tracked tool invocation with fallback. |
| <code>agentguard.tools.registry.get_registry()</code>          | Register tools/alternatives at startup.            |
| <code>agentguard.evaluators.base.Evaluator</code>              | Base class for a custom evaluator.                 |
| <code>agentguard.recovery.seed.seed_recovery_state(...)</code> | Seed the next run from a checkpoint.               |

| Name                                                                                                                                                                                                                           | What it does                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>agentguard.EvalResult</code> / <code>Decision</code> / <code>RiskAssessment</code> / <code>RootCause</code> / <code>HumanDecision</code> / <code>StateSnapshot</code> / <code>AgentState</code> / <code>RunStatus</code> | Typed data models you may read from run results, evaluators, or the dashboard API — you do not usually construct these yourself except <code>EvalResult</code> in a custom evaluator. |

---

*Generated from the current agentguard package source (`agentguard/__init__.py`, `client.py`, `decorator.py`, `context.py`, `models.py`, `errors.py`, `tools/registry.py`, `evaluators/base.py`, `recovery/seed.py`). Everything shown here is real, shipped, and covered by the project's automated test suite — no placeholder or planned-but-unbuilt functionality is included.*